

Autonomous OS Debugging Agent

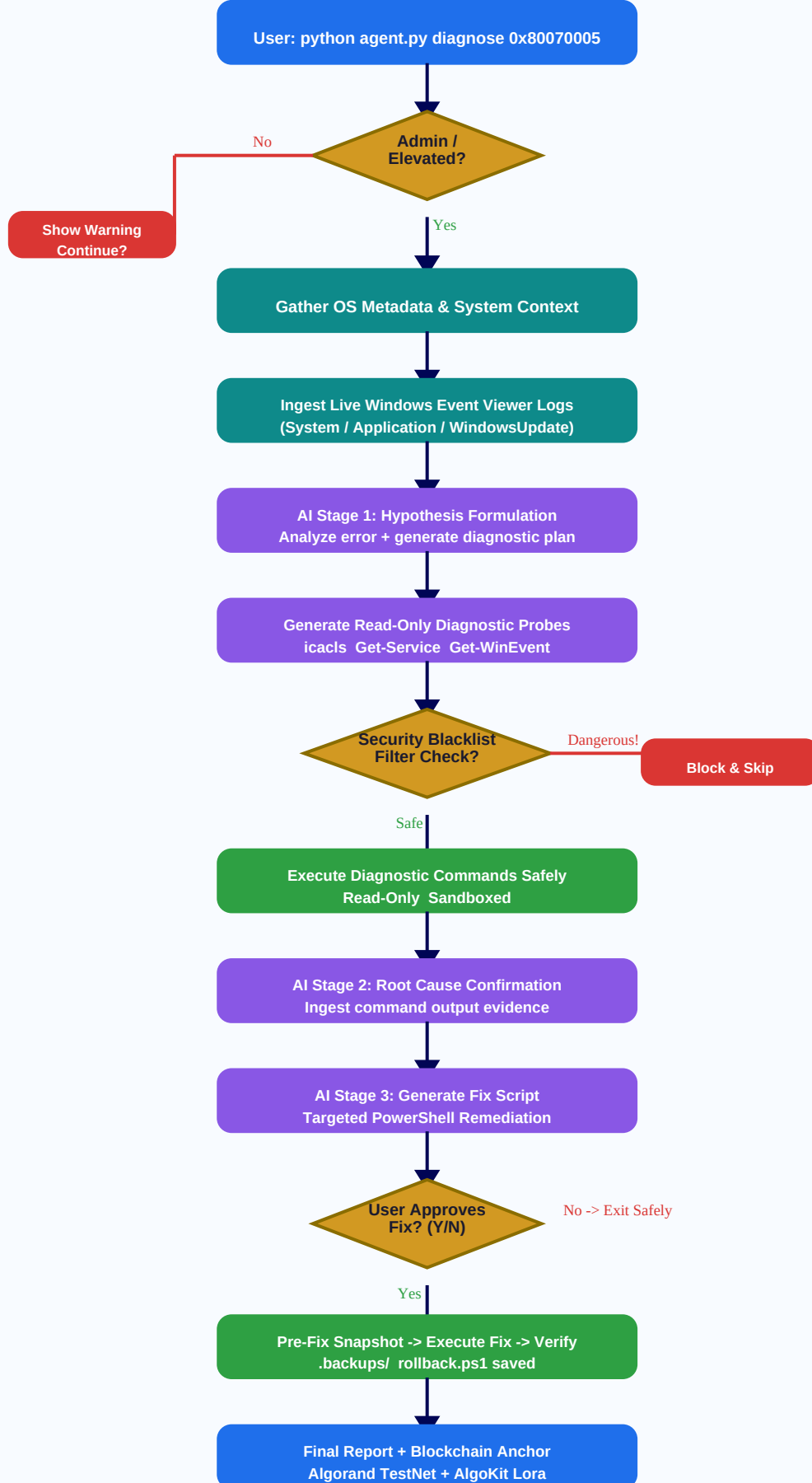
Complete Technical Reference: Flowcharts + 200 Q&A;

Team	KernelHealers
Members	Ansh (Leader) · Swati · Aman · Shivang
Hackathon Theme	Blockchain & Web3 — Agentic Solutions (x402)
Blockchain	Algorand TestNet AlgoKit Lora Explorer
GitHub	github.com/Ansh00031/SGU-AI-Thon

■ SECTION 1: System Workflow Flowchart

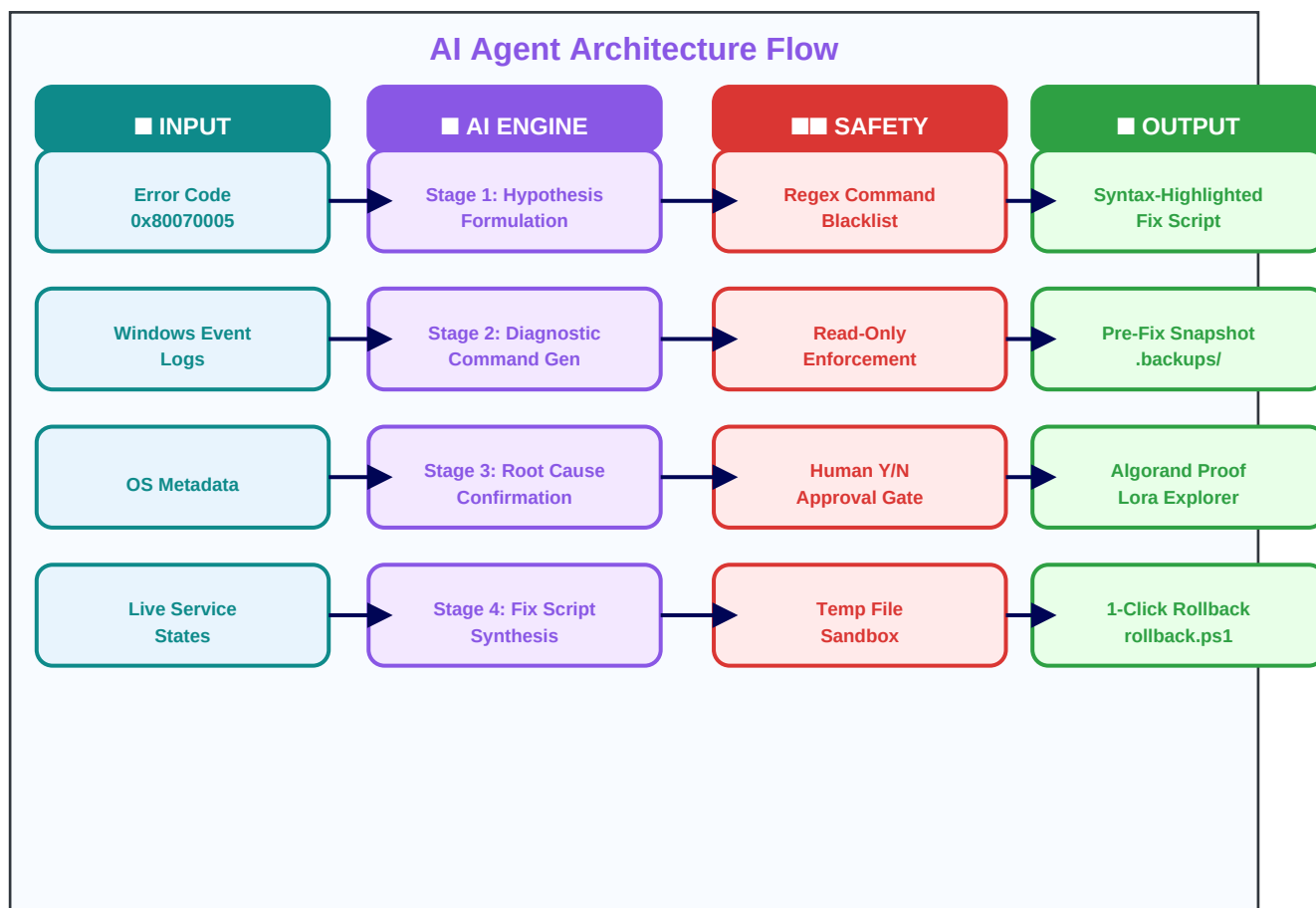
The flowchart below shows the complete end-to-end pipeline of the Autonomous OS Debugging Agent — from the moment a user runs the diagnose command to the final blockchain-anchored audit proof.

System Workflow Flowchart



SECTION 2: AI Agent Architecture

The AI Agent operates across 4 layers: Input (telemetry), AI Engine (5-stage reasoning), Safety (blacklist + human gate), and Output (fix, snapshot, blockchain proof, rollback).



SECTION 3: WinRE Cloud Recovery Flow

When a PC is completely unbootable (stuck in a boot loop), the agent can be downloaded and run entirely from the Windows Recovery Environment (WinRE) using a single command line.

WinRE Cloud Recovery Flow

■ PC Stuck — Boot Loop / Won't Start

User Boots into Windows Recovery (WinRE)

Opens WinRE Command Prompt

Runs: `irm ../bootstrap.ps1 | iex`

bootstrap.ps1 Downloads from GitHub

Detects Offline C:\ Windows Drive

Downloads Portable Python (~15MB)

Downloads agent.py + core/*.py

Auto-Installs: typer, rich, pydantic

■ Launches startup-monitor → Machine Diagnosed!

■ SECTION 4: 200 Important Questions

■ CATEGORY 1: Basic Project Understanding (Q1–Q20)

- Q1 What is the Autonomous OS Debugging Agent?
- Q2 What problem does this project solve?
- Q3 Why can't users just use ChatGPT to fix OS errors?
- Q4 What does the term 'autonomous' mean in this context?
- Q5 What is an OS error code like 0x80070005?
- Q6 What does '0x80070005: Access Denied' mean on Windows?
- Q7 Who is the target user for this tool?
- Q8 Is this project open-source? Where is the code?
- Q9 What operating systems does it support?
- Q10 What programming language is the project built in?
- Q11 What is the 'AI mechanic inside your terminal' analogy?
- Q12 How is this different from a standard antivirus tool?
- Q13 How long does a traditional IT support ticket take vs. your tool?
- Q14 What is the main CLI command to start diagnosis?
- Q15 Does the user need to know PowerShell to use this tool?
- Q16 What is the minimum Python version required?
- Q17 Does the tool require internet to work?
- Q18 Can it work fully offline?
- Q19 What is the GitHub repository link for this project?
- Q20 What was the inspiration behind building this?

■ CATEGORY 2: AI Engine & Reasoning Loop (Q21–Q50)

- Q21 What AI model powers the diagnostic reasoning?
- Q22 What is 'hallucination' in AI and why is it dangerous for OS repair?
- Q23 How does the agent avoid hallucinations?
- Q24 What is the Active Telemetry Reasoning Loop?
- Q25 What are the 5 stages of the AI reasoning pipeline?
- Q26 What is 'hypothesis formulation' in Stage 1?
- Q27 What are read-only diagnostic probes?

Q28	What is icacs and what does it check?
Q29	What is Get-Service used for in diagnostics?
Q30	What is Get-WinEvent and which logs does it read?
Q31	How does the AI confirm the root cause in Stage 3?
Q32	What information is fed back to the AI after probes run?
Q33	What is a 'remediation script' in Stage 4?
Q34	What language are remediation scripts written in?
Q35	How does the AI structure its JSON output for fix proposals?
Q36	What is the verification command in Stage 5?
Q37	Can the AI work without an OpenAI API key?
Q38	What happens in offline/heuristic mode without an API key?
Q39	Can you use a local LLM like Ollama or Llama 3.1?
Q40	How do you configure Ollama as the AI backend?
Q41	What is the OPENAI_BASE_URL setting used for?
Q42	What is the LLM_MODEL environment variable?
Q43	How does the agent handle LLM rate limits or timeouts?
Q44	What is a JSON Schema prompt and why use strict JSON?
Q45	What happens if the AI returns malformed JSON?
Q46	How does the agent ensure the AI stays on topic?
Q47	Can the AI diagnose Linux errors too?
Q48	What Windows Event Viewer channels does the agent read?
Q49	How many event log entries does it collect by default?
Q50	What is the --max-events flag used for?

■ CATEGORY 3: Security & Safety (Q51–Q80)

Q51	What is the Security Guardrail / Command Blacklist?
Q52	Which commands are blocked by the blacklist?
Q53	How is the blacklist implemented technically?
Q54	Can the blacklist be bypassed?
Q55	What is the Human-in-the-Loop (HITL) gate?
Q56	What exactly is shown to the user before executing a fix?
Q57	What happens if the user selects 'N' at the approval gate?

Q58	What is a 'read-only diagnostic probe' vs. a 'write command'?
Q59	How are diagnostic commands sandboxed?
Q60	What is the temp file sandbox execution model?
Q61	Are fix scripts auto-deleted after execution?
Q62	What is a pre-fix snapshot?
Q63	Where are snapshots stored on disk?
Q64	What does a snapshot contain?
Q65	What is a rollback script?
Q66	How is the rollback script generated?
Q67	How long does a rollback take to execute?
Q68	What is 'python agent.py rollback' and when do you use it?
Q69	Can you rollback multiple sessions?
Q70	What happens if the rollback script itself fails?
Q71	What is Administrator privilege and why is it required?
Q72	What is --skip-admin-check flag and when should it be used?
Q73	What is dry-run mode?
Q74	Can a standard (non-admin) user run the agent?
Q75	What are the most common dangerous commands blocked?
Q76	Does the agent access the internet during diagnosis?
Q77	Can the agent modify the Windows Registry?
Q78	What is the difference between diagnosis phase and remediation phase?
Q79	How does the agent handle timeouts on diagnostic commands?
Q80	What is the execute_diagnostic_command function?

■ CATEGORY 4: Sessions, Snapshots & Rollback (Q81–Q100)

Q81	What is a session ID and how is it generated?
Q82	Where are all sessions stored?
Q83	What is the .backups/ directory structure?
Q84	What files are inside a session's backup folder?
Q85	What is metadata.json in a session?
Q86	What is fix.ps1 in a session?
Q87	What is rollback.ps1 in a session?

Q88	What is the 'history' command used for?
Q89	How do you view all past sessions?
Q90	What session statuses exist? (COMPLETED, FAILED, ROLLED_BACK...)
Q91	What is VERIFIED_POST_REBOOT status?
Q92	What is blockchain_receipt.json inside a session?
Q93	Can you delete a session's backup manually?
Q94	What happens to the session if rollback is executed?
Q95	How many sessions are stored by default?
Q96	Is there a cleanup or expiry for old sessions?
Q97	What is the update_session_status function?
Q98	What is the get_session function?
Q99	What is the list_sessions function?
Q100	How does the rollback command find the most recent session?

■ CATEGORY 5: Auto-Start & Reboot Management (Q101–Q120)

Q101	What is the enable-autostart command?
Q102	Where is the startup batch file created on Windows?
Q103	What does the startup batch file do on boot?
Q104	What is the disable-autostart command?
Q105	What is the startup-monitor command?
Q106	What is the difference between Active Problems and Solved Problems?
Q107	What services does startup-monitor check live?
Q108	What is the startup-log command?
Q109	Where is the startup execution log stored?
Q110	What is the Windows RunOnce registry key?
Q111	What is register_reboot_hook used for?
Q112	What registry path is used for the RunOnce hook?
Q113	What command is registered in RunOnce after a fix?
Q114	What is python agent.py resume ?
Q115	When does the resume command fire automatically?
Q116	What is unregister_reboot_hook used for?
Q117	Why is it important to unregister the hook after it fires?

- | | |
|------|---|
| Q118 | What is <code>is_reboot_pending()</code> checking? |
| Q119 | Which Windows registry keys indicate a pending reboot? |
| Q120 | What happens if a reboot is needed but the user declines? |

■ CATEGORY 6: WinRE Cloud Recovery (Q121–Q140)

- | | |
|------|--|
| Q121 | What is Windows Recovery Environment (WinRE)? |
| Q122 | How do you enter WinRE on a Windows machine? |
| Q123 | What is the 1-line cloud rescue command? |
| Q124 | What is <code>bootstrap.ps1</code> and what does it do? |
| Q125 | What is <code>rescue.bat</code> used for? |
| Q126 | How does <code>bootstrap.ps1</code> find Python on a fresh machine? |
| Q127 | What is Portable Python and how is it downloaded? |
| Q128 | From where are the core agent files downloaded? |
| Q129 | What files does <code>bootstrap.ps1</code> download from GitHub? |
| Q130 | Does WinRE have internet access by default? |
| Q131 | How does <code>bootstrap.ps1</code> detect the offline Windows drive? |
| Q132 | What is <code>Invoke-WebRequest</code> used for in the bootstrap? |
| Q133 | What dependencies are installed by <code>bootstrap.ps1</code> ? |
| Q134 | What happens if <code>pip</code> is not available in WinRE? |
| Q135 | What is the <code>--user</code> fallback in <code>pip</code> installation? |
| Q136 | Can the agent fix a PC that won't boot at all? |
| Q137 | What types of errors can WinRE mode fix? |
| Q138 | What is DISM and how would it be used offline? |
| Q139 | What is the <code>rescue.bat</code> shortcut used for in WinRE? |
| Q140 | What is the network requirement for WinRE recovery? |

■ CATEGORY 7: Blockchain & Algorand (Q141–Q170)

- | | |
|------|--|
| Q141 | What is Algorand and why was it chosen? |
| Q142 | What is Algorand TestNet vs. MainNet? |
| Q143 | What is the AlgoKit Lora Explorer? |
| Q144 | What is the URL for AlgoKit Lora TestNet Explorer? |
| Q145 | What is an Algorand wallet and how is one generated? |

Q146	Where is the generated wallet stored securely?
Q147	What is ALGO and how much does a transaction cost?
Q148	What is a free TestNet ALGO faucet?
Q149	What is the Algorand Faucet URL used in the project?
Q150	What is a SHA-256 cryptographic hash?
Q151	What exactly is hashed in the audit proof?
Q152	What is an ARC-0002 structured note?
Q153	What JSON fields are included in the blockchain audit note?
Q154	What is a zero-ALGO self-transaction?
Q155	Why send a 0 ALGO transaction instead of a real transfer?
Q156	What is anchor_session_on_chain function?
Q157	What is get_or_create_wallet function?
Q158	What is get_wallet_balance function?
Q159	What is compute_session_hash function?
Q160	What is wait_for_confirmation in the blockchain module?
Q161	How long does it take to confirm a transaction on Algorand?
Q162	What is the Algorand block time?
Q163	What is testnet-api.algonode.cloud used for?
Q164	What Python library is used for Algorand interaction?
Q165	What is py-algorand-sdk?
Q166	What happens if the TestNet wallet has insufficient ALGO?
Q167	What is the --anchor flag in the diagnose command?
Q168	What is python agent.py blockchain status?
Q169	What is python agent.py blockchain anchor?
Q170	Can this be moved to Algorand MainNet for production?

■ CATEGORY 8: Enterprise & Real-World Impact (Q171–Q185)

Q171	What industries benefit most from this tool?
Q172	Why do enterprises need tamper-proof audit logs?
Q173	What compliance regulations require system change logs?
Q174	How does this tool help IT support teams?
Q175	How much time does the agent save per incident?

Q176	What is the cost of IT downtime for enterprises?
Q177	How does this tool prevent unauthorized system changes?
Q178	Can this be deployed across a fleet of enterprise PCs?
Q179	What is the business model for commercializing this?
Q180	How does this compare to existing tools like SolarWinds?
Q181	What is the competitive advantage of blockchain-backed logs?
Q182	Can this tool be integrated with ticketing systems like Jira?
Q183	What is the potential scale of this tool globally?
Q184	Can this handle Linux servers in future versions?
Q185	What are the current limitations of the project?

■ CATEGORY 9: Future Scope (Q186–Q195)

Q186	What is the next planned feature after this hackathon?
Q187	Will the tool support Linux and macOS?
Q188	What is auto patch discovery via WinGet?
Q189	Can the tool integrate with Microsoft Intune or SCCM?
Q190	What is an enterprise SaaS model for this tool?
Q191	Can the Algorand anchor move to MainNet for production use?
Q192	Will there be a GUI or web dashboard version?
Q193	Can the tool predict errors before they happen?
Q194	What is the plan for multi-language support?
Q195	Could this tool be integrated into a Windows 11 native widget?

■ CATEGORY 10: Judge Q&A; — Top 5 Critical Questions (Q196–Q200)

Q196	What if the AI gives a harmful command and deletes important files?
→	<i>Read-only probes run first. Every command is validated against a strict regex blacklist. Nothing runs without explicit Y/N user approval. And if anything goes wrong, 'python agent.py rollback' restores everything in under 1 second.</i>
Q197	Why not just use Windows built-in System Restore?
→	<i>System Restore doesn't tell you WHY something broke, doesn't use AI reasoning, doesn't generate verified fix scripts, and has no blockchain audit trail. Our tool diagnoses, explains, fixes, AND proves it on-chain.</i>
Q198	Is this tool safe to run on a production server?

→ *Yes — read-only probes first, blacklisted destructive commands, mandatory human Y/N approval, and pre-fix snapshots with 1-click rollback guarantee zero unintended risk.*

Q199 How is this different from just writing a PowerShell script yourself?

→ *You'd need to read logs manually, interpret output, write the correct targeted fix, test it safely, and handle rollback — all manually. Our tool does all of this autonomously, grounded in live telemetry, in 15 seconds.*

Q200 Why does an OS debugging tool need blockchain?

→ *Enterprise and government IT demands an immutable, third-party-verifiable audit trail. A local log file can be edited or deleted. An Algorand blockchain entry cannot be changed by anyone — ever.*

Team KernelHealers

Autonomous OS Debugging Agent — Blockchain & Web3 Track Build With 
2.0 Hackathon

github.com/Ansh00031/SGU-AI-Thon | lora.algokit.io/testnet